上海科技大学
ShanghaiTech University

CS240 Algorithm Design and Analysis

Lecture 11

NP and Computational Intractability

Quan Li
Fall 2025
2025.10.24

NP-Completeness

Polynomial Transformation

- **Def.** Problem X **polynomial reduces** (Cook) to problem y if arbitrary instances of problem X can be solved using:
  - Polynomial number of standard computational steps, plus
  - Polynomial number of calls to oracle that solves problem y

- **Def.** Problem X **polynomial transforms** (Karp) to problem Y if given any input x to X, we can construct an input y in poly-time such that x is a yes instance of X iff y is a yes instance of Y

We require |y| to be of size polynomial in |x|

- **Note.** Polynomial transformation is polynomial reduction with just one call to oracle for Y, exactly at the end of the algorithm for X. Almost all previous reductions were of this form
- **Open question.** Are these two concepts the same?

We abuse notation $\leq_p$ and blur distinction

NP-Complete

- **NP-complete.** A problem Y in NP with the property that for every problem X in NP, $X \leq_p Y$

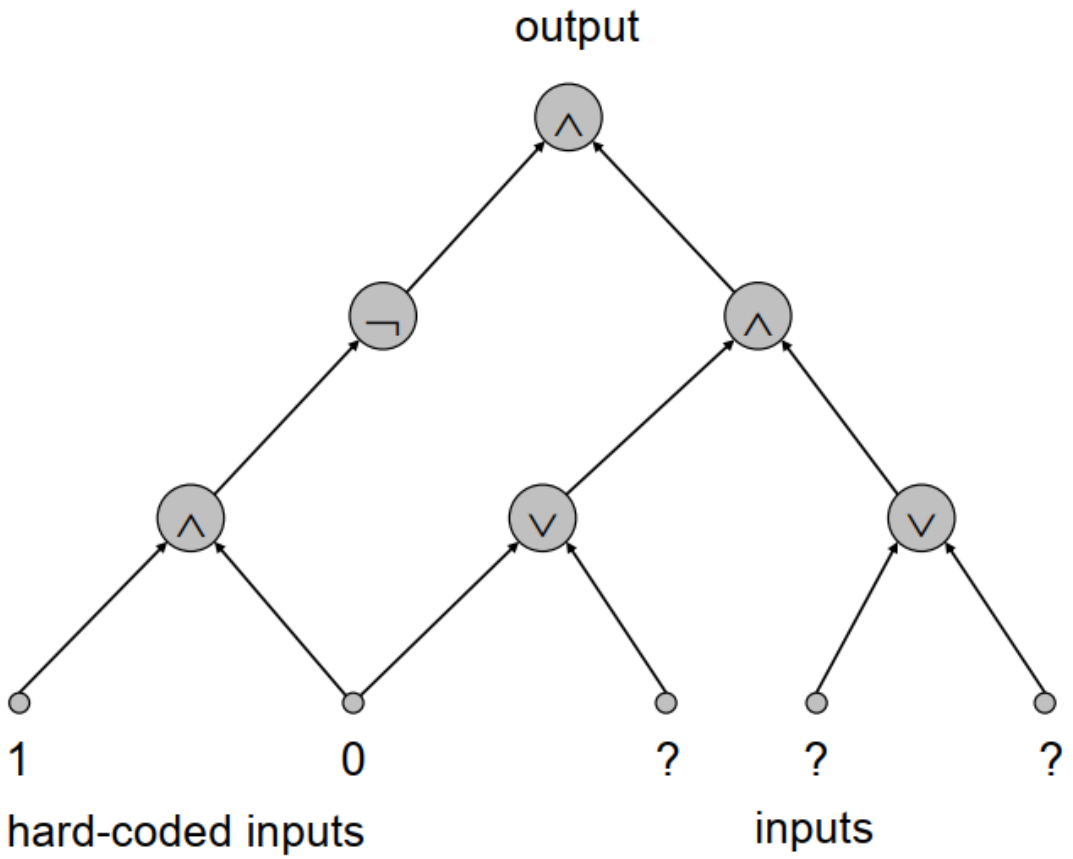
- **Theorem.** Suppose Y is an NP-complete problem. Then Y is solvable in poly-time iff P = NP
- **Pf.** ← If P = NP then Y can be solved in poly-time since Y is in NP
- **Pf.** → Suppose Y can be solved in poly-time
  - Let X be any problem in NP. Since $X \leq_p Y$ we can solve X in poly-time. This implies $NP \subseteq P$
  - We already know $P \subseteq NP$. Thus P = NP

- **Fundamental question.** Do there exist "natural" NP-complete problems?

Circuit Satisfiability

- **CIRCUIT-SAT.** Given a combinational circuit built out of AND, OR, and NOT gates, is there a way to set the circuit inputs so that the output is 1?

yes: 1 0 1



The "First" NP-Complete Problem

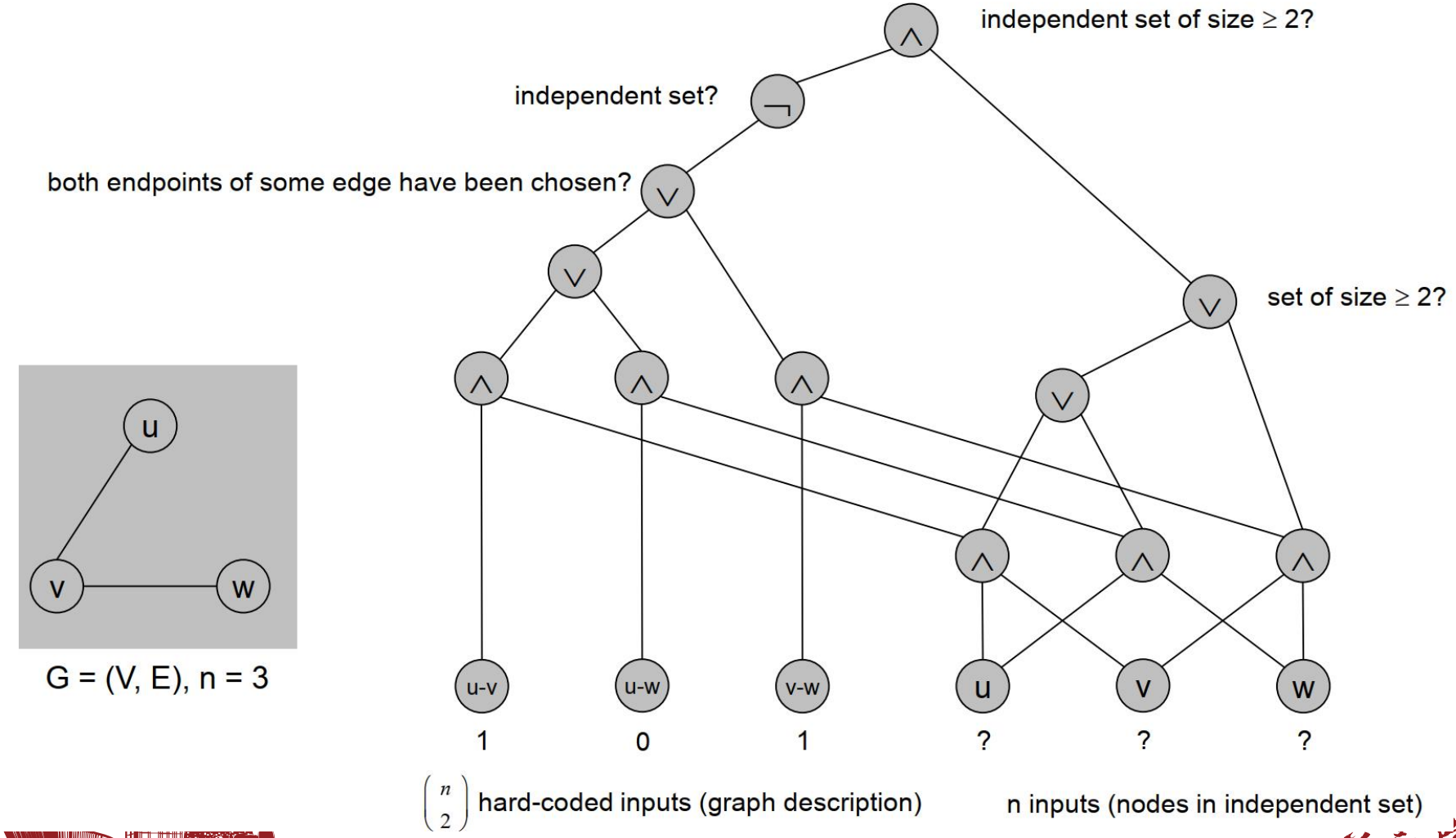
- **Theorem.** CIRCUIT-SAT is NP-complete. [Cook 1971, Levin 1973]
- **Pf.** (sketch)
  - Clearly, CIRCUIT-SAT is in NP
  - Any algorithm that takes a fixed number of bits n as input and produces a yes/no answer can be represented by such a circuit. Moreover, if algorithm takes poly-time, then circuit is of poly-size

Sketchy part of proof; fixing the number of bits is important, and reflects basic distinction between algorithms and circuits

  - Consider some problem X in NP. It has a poly-time certifier C(s, t). To determine whether s is in X, need to know if there exists a certificate t of length p(|s|) such that C(s, t) = yes
  - View C(s, t) as an algorithm on |s| + p(|s|) bits (input s, certificate t) and convert it into a poly-size circuit K
    - First |s| bits are hard-coded with s
    - Remaining p(|s|) bits represent (unknown) bits of t
  - Circuit K is satisfiable iff ∃t, C(s, t) = yes

Example

- **Ex.** Construction below creates as circuit K whose inputs can be set so that K outputs true iff graph G has an independent set of size ≥ 2



Establishing NP-Completeness

- **Remark.** Once we establish first "natural" NP-complete problem, others fall like dominoes
- **Recipe to establish NP-completeness of problem Y**
  - Step 1. Show that Y is in NP
  - Step 2. Choose an NP-complete problem X
  - Step 3. Prove that $X \leq_p Y$

- **Theorem.** If X is NP-complete problem and Y is a problem in NP with the property that $X \leq_p Y$, then Y is NP-complete
- **Pf.** Let W be any problem in NP. Then $W \leq_p X \leq_p Y$
  - By transitivity, $W \leq_p Y$
  - Hence Y is NP-complete

By definition of NP-complete

By assumption

3-SAT is NP-Complete

Review of 3-SAT

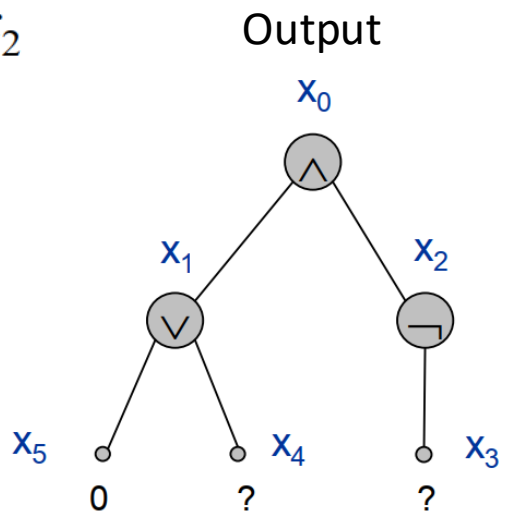
- **Literal:** A Boolean variable or its negation x_i or $\overline{x_i}$
- **Clause:** A disjunction of literals $C_j = x_1 \vee \overline{x_2} \vee x_3$
- **Conjunctive normal form:** A propositional formula **Φ** that is the conjunction of clauses $\Phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4$
- **SAT.** Given CNF formula **Φ**, does it have a satisfying truth assignment?
- **3-SAT:** SAT where each clause contains exactly 3 literals

Ex: $(\overline{x_1} \vee x_2 \vee x_3) \wedge (x_1 \vee \overline{x_2} \vee x_3) \wedge (\overline{x_1} \vee x_2 \vee x_4)$

Yes: x_1 = true, x_2 = true, x_3 = false, x_4 = false.

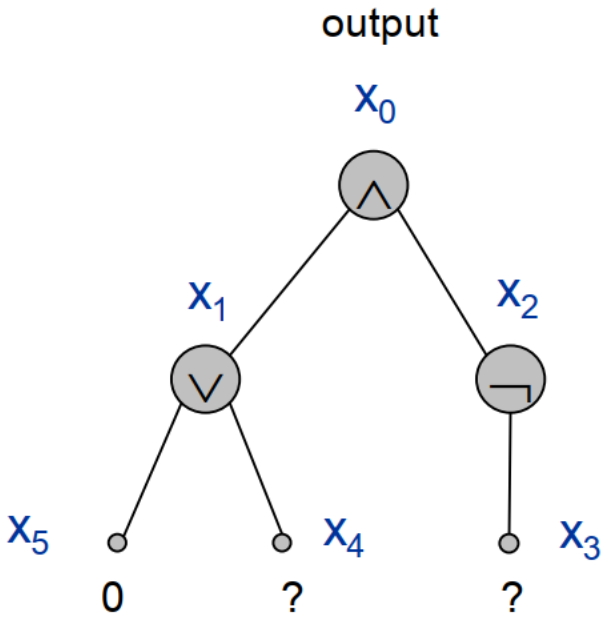
3-SAT is NP-Complete

- **Theorem.** 3-SAT is NP-complete
- **Pf.** Suffices to show that CIRCUIT-SAT $\leq_p$ 3-SAT since 3-SAT is in NP
  - Let K be any circuit. We can construct an instance of 3-SAT **Φ**
  - Step 1: Create a 3-SAT variable x_i for each circuit element i
  - Step 2: Make circuit compute correct values at each node:
    - $X_2 = \neg X_3$ → add 2 clauses: $x_2 \vee x_3$, $\overline{x_2} \vee \overline{x_3}$
    - $X_1 = X_4 \vee X_5$ → add 3 clauses: $x_1 \vee \overline{x_4}$, $x_1 \vee \overline{x_5}$, $\overline{x_1} \vee x_4 \vee x_5$
    - $X_0 = X_1 \wedge X_2$ → add 3 clauses: $\overline{x_0} \vee x_1$, $\overline{x_0} \vee x_2$, $x_0 \vee \overline{x_1} \vee \overline{x_2}$
  - Step 3: Hard-coded input values and output value
    - $X_5 = 0$ → add 1 clauses: $\overline{x_5}$
    - $X_0 = 1$ → add 1 clauses: x_0
  - Final step: turn clauses of length < 3 into clauses of length exactly 3



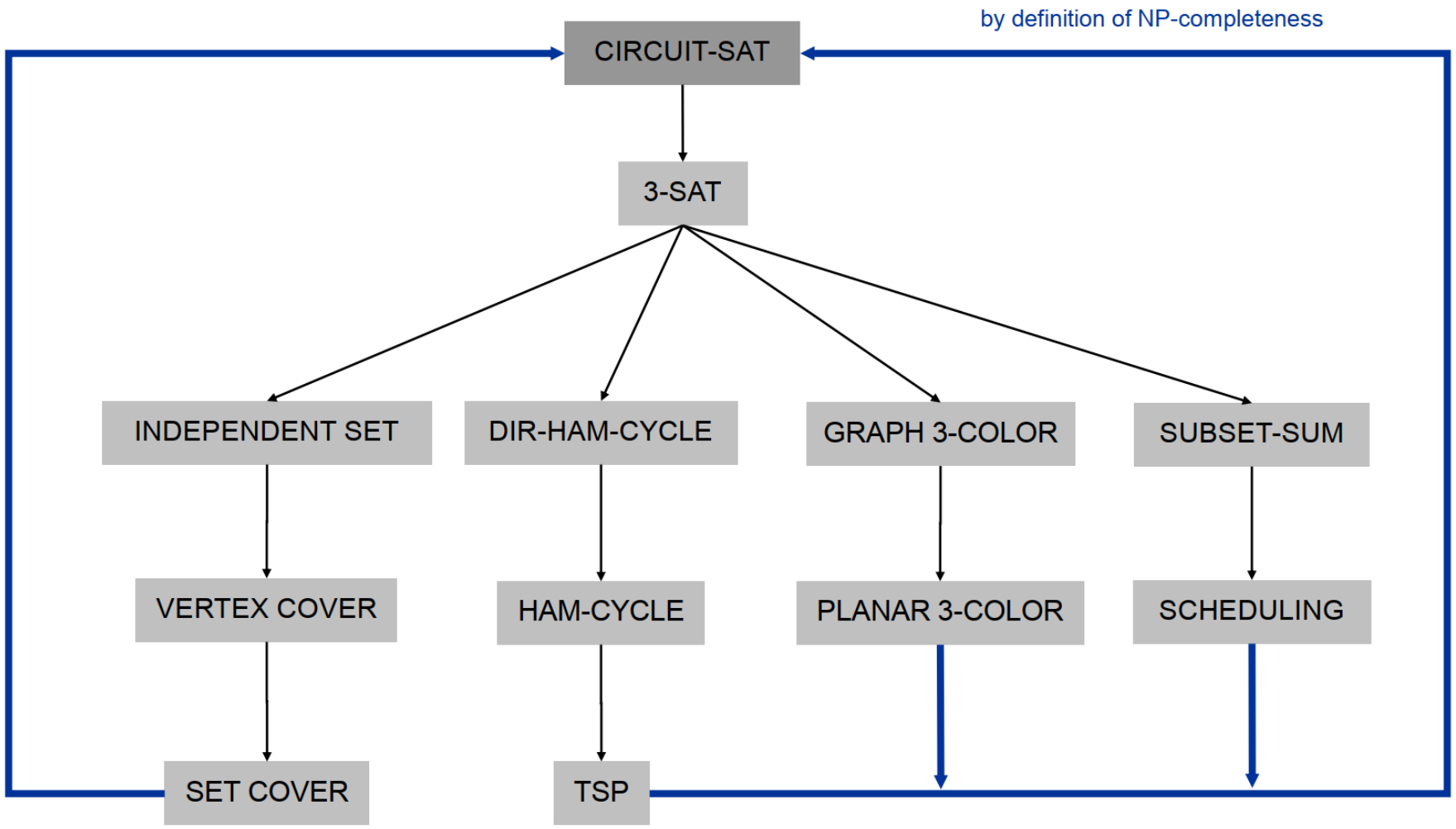
3-SAT is NP-Complete

- **Lemma.** ϕ is satisfiable iff the inputs of K can be set s.t. it outputs 1
- **Pf. ←**
  - Suppose there are inputs of K s.t. it outputs 1
  - Compute the values of all the nodes in K
  - According to our construction, this set of values satisfies ϕ
- **→**
  - Suppose ϕ is satisfiable
  - The variable assignment of ϕ specifies the values of all the nodes in K
  - Our construction ensures that all the nodes in K are correctly computed and the output is 1



NP-Completeness

- **Observation.** All problems below are NP-complete and polynomial reduce to one another!



Some NP-Complete Problems

- **Six basic genres of NP-complete problems and paradigmatic examples**
  - Packing problems: SET-PACKING, INDEPENDENT SET
  - Covering problems: SET-COVER, VERTEX-COVER
  - Constraint satisfaction problems: SAT, 3-SAT
  - Sequencing problems: HAMILTONIAN-CYCLE, TSP
  - Partitioning problems: 3D-MATCHING, 3-COLOR
  - Numerical problems: SUBSET-SUM, KNAPSACK

- **Practice.** Most NP problems are known to be either in P or NP-complete

- **Notable exceptions**. Factoring, graph isomorphism, Nash equilibrium
- **Theorem.** [Ladner 1975] Unless P = NP, there exist problems in NP that are neither in P nor NP-complete

Extent and Impact of NP-Completeness

- **Extent of NP-completeness.** [Papadimitriou 1995]
  - 6,000 citations per year (title, abstract, keywords)
  - More than “compiler”, “operating system”, “database”
  - Prime intellectual export of CS to other disciplines
  - Broad applicability and classification power
  - “Captures vast domains of computational, scientific, mathematical endeavors, and seems to roughly delimit what mathematicians and scientists had been aspiring to compute feasibly”

- **NP-completeness can guide scientific inquiry**
  - 1926: Ising introduces simple model for phase transitions
  - 1944: Onsager finds closed-form solution to 2D-Ising
  - 19xx: Feynman and other top minds seek 3D solution
  - 2000: Istrial proves 3D-Ising is NP-complete; thus search for closed-form solution to 3D-Ising appears doomed

More Hard Computational Problems

- **Aerospace engineering:** optimal mesh partitioning for finite elements.
- **Biology:** protein folding.
- **Chemical engineering:** heat exchanger network synthesis.
- **Civil engineering:** equilibrium of urban traffic flow.
- **Economics:** computation of arbitrage in financial markets with friction.
- **Electrical engineering:** VLSI layout.
- **Environmental engineering:** optimal placement of contaminant sensors.
- **Financial engineering:** find minimum risk portfolio of given return.
- **Game theory:** find Nash equilibrium that maximizes social welfare.
- **Genomics:** phylogeny reconstruction.
- **Mechanical engineering:** structure of turbulence in sheared flows.
- **Medicine:** reconstructing 3-D shape from biplane angiocardiogram.
- **Operations research:** optimal resource allocation.
- **Physics:** partition function of 3-D Ising model in statistical mechanics.
- **Politics:** Shapley-Shubik voting power.
- **Pop culture:** Minesweeper consistency.
- **Statistics:** optimal experimental design.

Terminology

- **NP-complete.** A problem in NP such that every problem in NP polynomial reduces to it
- **NP-hard.** [Bell Labs, Steve Cook, Ron Rivest, Sartaj Sahni] A problem such that every problem in NP reduces to it

Stephen Cook

Stephen Cook

- 1971 paper "The Complexity of Theorem Proving Procedures"
  - formalized the notions of polynomial-time reduction and NP-completeness
  - proved the existence of an NP-complete problem (SAT)
- Bio
  - Joined UC Berkeley in 1966 but was denied tenure in 1970
  - Joined U of Toronto in 1970
  - 1982 Turing award

Richard Karp

"It is to our everlasting shame that we were unable to persuade the math department to give him tenure." -- Richard Karp (1985 Turing award)

Next Time:
NP and Computational Intractability (Cont.)